

Bank Marketing Campaign

Week - 4

Name : Nandita Mori
Roll number : 467

Model Evaluation

Step 1. Import the necessary libraries

```
In [5]: import pandas as pd
import numpy as np
import joblib

from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    confusion_matrix
)
```

Step 2. Assign dataset to a variable called df

```
In [6]: df = pd.read_csv("../Dataset/bank-full.csv", sep=";")
df
```

Out[6]:

	age	job	marital	education	default	balance	housing	loan	contact
0	58	management	married	tertiary	no	2143	yes	no	unknown
1	44	technician	single	secondary	no	29	yes	no	unknown
2	33	entrepreneur	married	secondary	no	2	yes	yes	unknown
3	47	blue-collar	married	unknown	no	1506	yes	no	unknown
4	33	unknown	single	unknown	no	1	no	no	unknown
...	...	...	...	...	...	...	...	...	...
45206	51	technician	married	tertiary	no	825	no	no	cellular
45207	71	retired	divorced	primary	no	1729	no	no	cellular
45208	72	retired	married	secondary	no	5715	no	no	cellular
45209	57	blue-collar	married	secondary	no	668	no	no	telephone
45210	37	entrepreneur	married	secondary	no	2971	no	no	cellular

45211 rows × 17 columns



Step 3: Separate X and y

```
In [7]: X = df.drop('y', axis=1)
        y = df['y']
```

Step 4: Convert target yes/no into 1/0

```
In [8]: y = y.map({'no': 0, 'yes': 1})
```

Step 5: Identify numerical and categorical columns

```
In [9]: numeric_columns = [
        'age',
        'balance',
        'day',
        'duration',
        'campaign',
        'pdays',
        'previous'
        ]

print("Numerical columns:")
print(numeric_columns)
```

```
Numerical columns:
['age', 'balance', 'day', 'duration', 'campaign', 'pdays', 'previous']
```

```
In [11]: categorical_columns = X.select_dtypes(  
        include=['object']  
        ).columns.tolist()  
  
        print("\nCategorical columns:")  
        print(categorical_columns)
```

Categorical columns:

['job', 'marital', 'education', 'default', 'housing', 'loan', 'contact', 'month', 'p
outcome']

Step 6: Convert categorical columns into numbers

```
In [12]: X = pd.get_dummies(X, columns=categorical_columns, drop_first=True)
```

```
In [13]: X = X.astype(int)
```

Step 7: Split the data into training and testing

```
In [10]: X_train, X_test, y_train, y_test = train_test_split(  
        X,  
        y,  
        test_size=0.2,  
        random_state=42,  
        stratify=y  
        )
```

Step 8: Scale numerical features

```
In [11]: scaler = StandardScaler()  
  
        X_train[numeric_columns] = scaler.fit_transform(  
        X_train[numeric_columns]  
        )  
  
        X_test[numeric_columns] = scaler.transform(  
        X_test[numeric_columns]  
        )
```

```
In [12]: X_train
```

Out[12]:

	age	balance	day	duration	campaign	pdays	previous	job_blue coll:
24001	-0.460434	-0.164410	1.582124	-0.457311	-0.246104	-0.410910	-0.241509	
43409	-1.589641	0.899627	-1.298384	2.502493	0.398202	1.446096	2.664584	
20669	0.292371	-0.365486	-0.458236	5.697691	0.398202	-0.410910	-0.241509	
18810	0.668773	-0.445003	1.822166	-0.862499	2.653271	-0.410910	-0.241509	
23130	-0.272233	-0.361249	1.222060	-0.777602	2.331118	-0.410910	-0.241509	
...	...	...	...	...	...	...	...	
17958	0.856974	-0.146160	1.702145	-0.773743	-0.246104	-0.410910	-0.241509	
15941	-0.460434	-0.437834	0.741975	-0.700423	0.720354	-0.410910	-0.241509	
16952	0.386471	-0.419258	1.102039	-0.619386	-0.568256	-0.410910	-0.241509	
34781	-1.307339	0.388954	-1.178363	-0.121583	-0.568256	-0.410910	-0.241509	
44229	1.797980	-0.289227	0.381912	-0.329966	-0.246104	2.704068	3.079741	

36168 rows × 42 columns



Step 9 : Load Existing Model(.pkl file)

```
In [13]: model = joblib.load("model.pkl")

print("Existing model loaded successfully!")
print(model)
```

Existing model loaded successfully!
LogisticRegression(max_iter=1000)

Step 10 : Make Predictions Using Existing model

```
In [16]: y_pred = model.predict(X_test)

print("\nFirst 20 predictions:")
print(y_pred[:20])
```

First 20 predictions:
[0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0]

Step 11 : Model Evaluation

```
In [17]: accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
```

```
print("Model Evaluation")
print("-----")
print("Accuracy :", accuracy)
print("Precision:", precision)
print("Recall   :", recall)
print("F1 Score :", f1)
```

Model Evaluation

```
-----
Accuracy : 0.9015813336282207
Precision: 0.6473684210526316
Recall   : 0.3487712665406427
F1 Score : 0.4533169533169533
```

Step 12 : Classification Report

```
In [20]: print("Classification Report")
print("-----")

print(classification_report(
    y_test,
    y_pred,
    target_names=["No", "Yes"]
))
```

Classification Report

```
-----
              precision    recall  f1-score   support

     No         0.92        0.97        0.95        7985
     Yes         0.65        0.35        0.45        1058

 accuracy          0.90          0.90        0.90        9043
  macro avg         0.78        0.66        0.70        9043
 weighted avg         0.89        0.90        0.89        9043
```

Step 13 : Confusion Matrix

```
In [21]: cm = confusion_matrix(y_test, y_pred)

print("Confusion Matrix:")
print(cm)
```

Confusion Matrix:

```
[[7784  201]
 [ 689 369]]
```

Step 14 : Check for Underfitting / Overfitting

```
In [23]: train_score = model.score(X_train, y_train)
test_score = model.score(X_test, y_test)

print("Overfitting / Underfitting Check")
```

```
print("-----")
print("Training Accuracy:", train_score)
print("Testing Accuracy :", test_score)

difference = train_score - test_score

print("Difference      :", difference)
```

Overfitting / Underfitting Check

Training Accuracy: 0.9020128290201282

Testing Accuracy : 0.9015813336282207

Difference : 0.0004314953919075659

```
In [24]: if train_score > 0.90 and difference > 0.10:
          print("Result: Model may be overfitting.")

          elif train_score < 0.70 and test_score < 0.70:
          print("Result: Model may be underfitting.")

          else:
          print("Result: Model appears to be fitting reasonably well.")
```

Result: Model appears to be fitting reasonably well.

In []: